Operating Systems: Principles, Processes & Concurrency

CST305: Operating Systems (S5 CSE/CET)

Prepared for Semester 5 Examination

College of Engineering Trivandrum (CET)

Author: **Sudheesh** | Compiled: August 19, 2026

Contents

1 Introduction to Operating Systems 2
1.1 Definition and Role of an Operating System 2
1.2 Abstract System Architecture 2
1.3 Fundamental Components of an Operating System 2
2 Core Functions of an Operating System 3
2.1 1. Process Management 3
2.2 2. Memory Management 3
2.3 3. File System Management 4
2.4 4. Device Management (I/O System) 4
2.5 5. Protection and Security 5
3 Process Concept, Lifecycle, and Management 5
3.1 Process Memory Address Space 5
3.2 Process States and 5-State Transition Model 6
3.3 The Process Control Block (PCB) 6
3.4 Operations on Processes: Creation and Termination 7
4 Process Schedulers 8
5 Threads and Multithreading 9
5.1 Concept of Threads 9
5.2 Benefits and Need for Threads 9
5.3 User-Level Threads (ULT) vs Kernel-Level Threads (KLT) 10
6 Process Synchronization & The Critical Section Problem 10
6.1 Independent vs Cooperative Processes 10
6.2 The Critical Section Problem 10
6.3 Three Mandatory Criteria for a Valid CS Solution 11
7 Software Solutions for Two Processes 11
7.1 Algorithm 1: Strict Alternation (Turn Variable) 11
7.2 Algorithm 2: Flag Array (Signaling Intent) 12
7.3 Algorithm 3: Peterson's Algorithm 12
8 Inter-Process Communication (IPC) 13
8.1 Overview and Comparison of IPC Models 13
8.2 Message Passing Operations and Communication Link Properties 14
8.3 Buffering Capacities in Message Passing 14
9 Summary Review & Exam Booster Cheat Sheet 15

1 Introduction to Operating Systems

1.1 Definition and Role of an Operating System

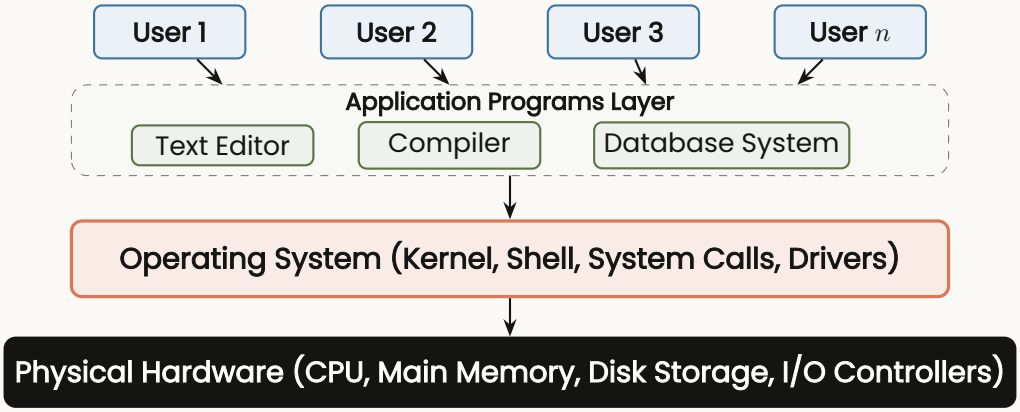
Operating System (OS): An **Operating System** is system software that manages and handles both hardware and software resources of a computing device, acting as an intermediary and abstract interface between the user/applications and the underlying physical hardware.

An Operating System operates as a **Resource Manager** and an **Extended Machine**:

- **Resource Management:** Controls and allocates computing resources such as the Central Processing Unit (CPU), primary/secondary memory (RAM, disk), and file storage.
- **User-Hardware Interface:** Provides an abstraction layer hiding the raw, complex hardware details (registers, interrupt controllers, buses) and presenting a clean, convenient execution environment.
- **Core System Services:** Executes foundational management functions including process lifecycle handling, memory allocation, device communication, file systems, and system protection.
- **Organizational Analogy:** Organizes and coordinates diverse system resources much like government ministries or administrative departments manage state infrastructure.
- **Prominent Real-World Examples:** Linux, UNIX, Microsoft Windows 11, MS-DOS, Android, Apple macOS, iOS.

1.2 Abstract System Architecture

A computing system is conceptually organized into hierarchical layers. The OS resides between user application programs and physical hardware, ensuring isolated and governed execution.



1.3 Fundamental Components of an Operating System

An Operating System is broadly structured into two primary operational entities:

1. The Shell (User Interface / Command Interpreter)

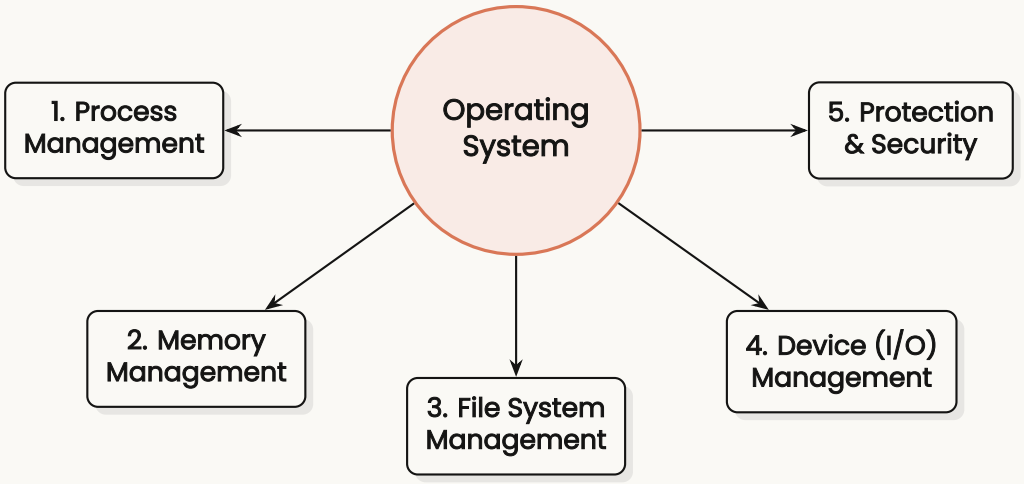
- The **outermost layer** of the operating system responsible for handling direct user interaction.
- Interprets user commands entered via a Command Line Interface (CLI) such as `bash`, `zsh`, or graphical gestures in a Graphical User Interface (GUI).
- Translates user commands into underlying system calls dispatched to the kernel, and formats output returned by the kernel for display.

2. The Kernel (Core Engine)

- The **central core component** that resides in main memory continuously from boot time.
- Operates in privileged mode (kernel/supervisor mode) with unrestricted hardware access.
- Acts as the primary interface between application software and low-level hardware devices, managing CPU scheduling, page tables, hardware interrupts, and device drivers.

2 Core Functions of an Operating System

The operating system coordinates multi-program execution, conceals complex hardware specifics through abstraction, enforces multi-tenant security, and maximizes hardware utilization.



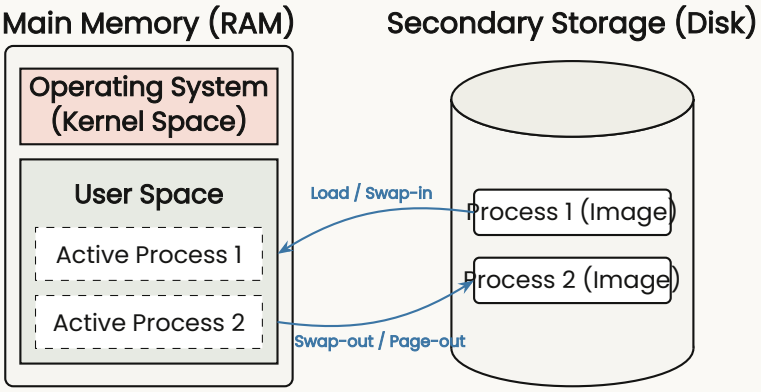
2.1 1. Process Management

A **process** is a program in active execution. Process management oversees process execution, concurrency, resource contention, and lifecycle transitions.

- **CPU Scheduling:** Dynamically decides which ready process is allocated the CPU, choosing algorithms based on throughput, response time, and fairness.
- **Process Synchronization:** Controls concurrent access to shared mutable data using locks, semaphores, and monitors to prevent race conditions and non-deterministic execution.
- **Deadlock Handling:** Employs prevention, avoidance (Banker's algorithm), detection, and recovery schemes to resolve resource lockups where processes wait indefinitely for each other.
- **Inter-Process Communication (IPC):** Provides structured protocols (shared memory or message queues) allowing cooperative processes to exchange signals and data.

2.2 2. Memory Management

Memory management regulates how data is organized across the physical memory hierarchy (high-speed registers, cache, main memory/RAM, and secondary magnetic/solid-state storage).



Key Memory Management Activities

1. **Dynamic Allocation & Deallocation:** Allocates memory blocks dynamically upon request (`malloc()`, `mmap()`) and reclaims space when released (`free()`).
2. **Memory Protection:** Enforces boundary checking via base and limit registers to prevent unauthorized processes from reading or overwriting other processes or kernel addresses.
3. **Virtual Memory:** Maps contiguous virtual address spaces onto non-contiguous physical pages, utilizing backing disk swap space to execute programs larger than physical RAM.
4. **Fragmentation Handling:** Minimizes internal and external fragmentation via dynamic relocation, paging, segmentation, and compaction.
5. **Secondary Storage & Disk Scheduling:** Organizes file blocks, tracks free-space bit vectors, and optimizes seek times using algorithms (FCFS, SSTF, SCAN/C-SCAN).

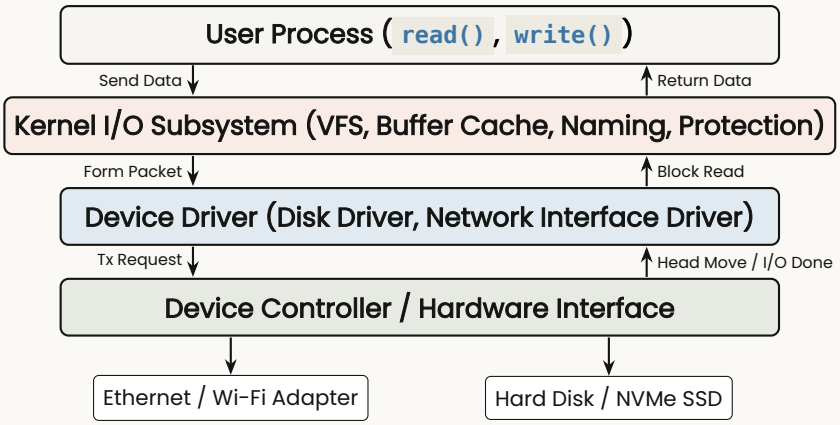
2.3 3. File System Management

> **File System:** A collection of access methods, directory structures, and storage management mechanisms allowing uniform, persistent, named access to non-volatile secondary media.

- **File Attributes:** Name, identifier, type, physical location, size, protection permissions (read / write / execute), and timestamps (creation, last access, modification).
- **File Types:** Regular files (ASCII text, binary objects, executables), directory files, character/block device files, named pipes (FIFOs), and symbolic links.
- **Primitive Operations:** Create, Open, Read, Write, Seek (reposition), Truncate, Delete, and Close.
- **File Access Methods:**
  - *Sequential Access:* Information processed in order, byte by byte (e.g., tape drives, standard stream pipes).
  - *Direct (Random) Access:* Blocks read/written in arbitrary order by fixed logical record numbers (e.g., database tables).
  - *Indexed Access:* Uses an auxiliary index structure (e.g., B-Tree) for rapid keyed searches.

2.4 4. Device Management (I/O System)

The I/O subsystem isolates hardware-specific quirks from the rest of the kernel, presenting a uniform device-driver interface.



Major Mechanisms in I/O Management

- **Device Drivers:** Dedicated translation software routines that convert generic I/O requests (`read_block`) into hardware-specific controller register commands.
- **Buffering:** Temporary memory areas used to smooth speed mismatches between fast CPUs and slow peripheral devices, or reconcile differing data transfer sizes.
- **Caching:** Retains frequently referenced disk blocks in high-speed main memory to avoid repetitive physical I/O transfers.
- **Spooling (Simultaneous Peripheral Operations On-Line):** Intercepts data intended for dedicated, unshareable devices (e.g., printers). The OS buffers all print jobs onto disk, allowing the generating processes to finish immediately while the spooler daemon feeds the printer sequentially.

2.5 5. Protection and Security

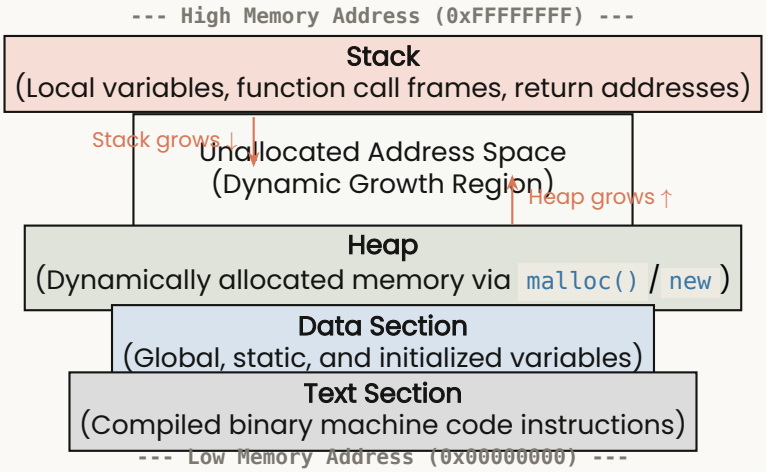
> **Protection vs Security:** **Protection** refers to internal mechanisms controlling the access of programs, processes, or users to system resources. **Security** encompasses external and internal defenses safeguarding system integrity against unauthorized intrusion, data corruption, and malicious attacks.

- **Access Control:** Assigns privilege levels (Access Control Lists, POSIX `rwx` bits) governing whether an entity may read, write, or execute a resource.
- **Authentication:** Positively validates user identity via identifiers (UID, SID), passwords, security tokens, or biometric parameters.
- **Resource Protection:** CPU dual-mode execution (User Mode vs Kernel Mode), memory protection keys, and I/O privilege isolation.
- **System Security:** Defends against malware, privilege escalation, Denial of Service (DoS/DDoS), and illicit data exfiltration.

3 Process Concept, Lifecycle, and Management

3.1 Process Memory Address Space

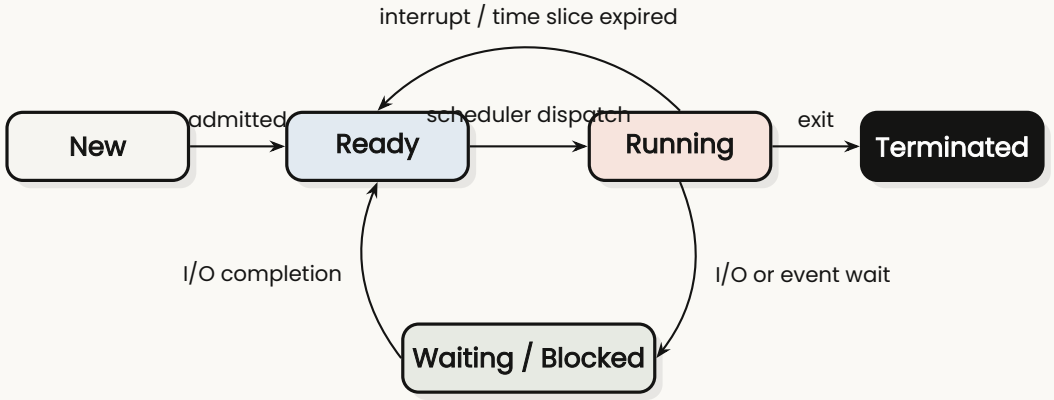
A process is more than passive program code (the text section). It represents an active execution instance containing program counters, registers, and specific memory segments.



3.2 Process States and 5-State Transition Model

During its existence, a process moves through discrete execution states:

1. **New:** The process is actively being created and initialized by the OS.
2. **Ready:** The process is loaded in main memory, waiting to be allocated CPU time.
3. **Running:** Instructions belonging to the process are being executed on the CPU.
4. **Waiting (Blocked):** The process cannot execute until an external event occurs (e.g., I/O completion, timer signal, or lock acquisition).
5. **Terminated:** The process has finished execution and its resources are released.



3.3 The Process Control Block (PCB)

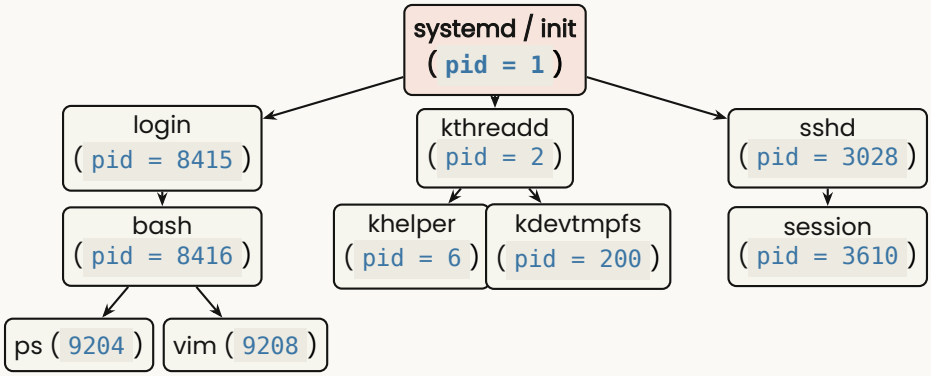
The **Process Control Block** (or Task Control Block) is the primary data structure maintained in kernel space for every active process.

PCB Field	Description and Function
Pointer	Linked list pointer linking the PCB into scheduling / device queues.
Process State	Current lifecycle state (`New`, `Ready`, `Running`, `Waiting`, `Terminated`).
Process ID (PID)	Unique positive integer identifying the process within the system.
Program Counter (PC)	Address of the next binary instruction to be fetched and executed.
CPU Registers	Context save area for accumulators, index registers, stack pointers, and flags.
Memory Limits	Base/limit register values, page tables, or segment descriptors.
Scheduling Info	Process priority, scheduling queue pointers, and aging counters.
Accounting Info	CPU time consumed, wall-clock limits, account numbers, and thread counts.
I/O Status Info	List of allocated I/O devices, open file descriptors, and pending operations.

3.4 Operations on Processes: Creation and Termination

1. Process Creation and UNIX Process Trees

Operating systems create processes hierarchically. The initial creator is the **parent process**, and created instances are **child processes**, forming a directed tree rooted at `systemd` or `init` (PID = 1).



Process Creation Options

- **Execution Alternatives:**
  1. Parent executes concurrently with its child.
  2. Parent blocks and waits (`wait()`) until some or all child processes terminate.

- **Address Space Alternatives:**
  1. Child is an exact duplicate of the parent (inherits memory pages, open files via Copy-on-Write).
  2. Child has a new program loaded into its address space using `exec()`.
- **Resource Partitioning:** Child obtains resources directly from the OS, or is restricted to a subset of parent resources to prevent runaway resource exhaustion.

UNIX Process Lifecycle using fork(), exec(), wait(), and exit()

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork(); // Fork returns 0 to child, child PID to parent

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    } else if (pid == 0) {
        // Child Process
        printf("Child executing command...\n");
        execlp("/bin/ls", "ls", "-l", NULL); // Replaces address space
        _exit(0); // Called if exec fails
    } else {
        // Parent Process
        int status;
        wait(&status); // Blocks until child terminates
        printf("Parent resumed after child completion with status %d\n",
    status);
    }
    return 0;
}
```

2. Process Termination and Cascading Termination

- A process concludes by executing `exit(status)`, returning an integer exit status to its waiting parent.
- Operating systems reclaim all allocated physical/virtual memory, open file tables, and I/O buffers.
- **Cascading Termination:** In many OS environments, if a parent process terminates (normally or abnormally), all its child and descendant processes are forcefully terminated by the OS to prevent orphaned zombie trees.

4 Process Schedulers

A **Scheduler** is an OS component that selects processes from memory/disk queues and assigns CPU time to balance utilization, throughput, and response latency.

Scheduler Type	Alternative Name	Frequency & Speed	Core Responsibility
Long-Term	Job Scheduler	Infrequent (seconds/mins), Slow	Controls Degree of Multiprogramming; loads jobs from disk spool into RAM.
Short-Term	CPU Scheduler	Frequent (10–100 ms), Extremely Fast	Selects next ready process in RAM and assigns CPU core.
Medium-Term	Swapper	Intermediate frequency	Swaps blocked/idle processes to disk to relieve RAM pressure; swaps in when ready.

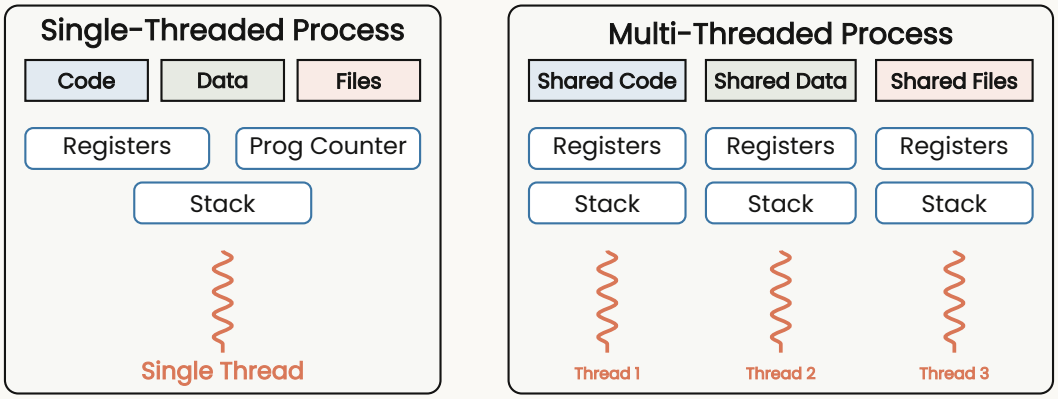
Job Mix Balance

The Long-Term Scheduler must maintain a balanced mix of **CPU-bound processes** (which spend most of their time computing) and **I/O-bound processes** (which spend most of their time waiting for I/O). If all jobs are I/O-bound, the ready queue is empty and the CPU sits idle; if all are CPU-bound, the I/O queue sits idle.

5 Threads and Multithreading

5.1 Concept of Threads

Thread: A **Thread** (often called a *Lightweight Process* or LWP) is the basic unit of CPU utilization. It comprises a unique Thread ID, a Program Counter (PC), a private Register Set, and an isolated Stack space, while sharing its Code section, Data section, and OS resources (e.g., open files) with peer threads of the same parent process.



5.2 Benefits and Need for Threads

1. **Enhanced Responsiveness:** In interactive GUI applications, a background thread handles heavy file I/O or network fetching while the UI thread remains completely responsive to user clicks.
2. **Resource Sharing:** Peer threads naturally share the process's address space and open file table without requiring IPC setups.

3. **Economy and Low Overhead:** Creating a thread is roughly 30× cheaper than creating a separate process; context switching between threads avoids expensive TLB/page-table invalidations.
4. **Scalability on Multicore Systems:** Threads run simultaneously on multiple hardware processor cores, achieving genuine hardware parallelism.

5.3 User-Level Threads (ULT) vs Kernel-Level Threads (KLT)

Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Management Layer	Implemented entirely by user-space thread libraries (e.g., POSIX pthreads, green threads).	Managed directly by the OS Kernel via system calls.
Kernel Awareness	Kernel is unaware of individual threads; sees only the parent process.	Kernel maintains thread tables, scheduling individual threads across cores.
Switching Speed	Extremely fast; no mode switch to kernel required.	Slower; requires user-to-kernel mode switch overhead.
Blocking Calls	If one thread executes a blocking system call, the **entire process blocks**.	A blocking system call in one thread does **not** block peer threads.
Multiprocessing	Cannot exploit multi-core CPUs; all threads share the process's assigned core.	Kernel schedules different threads onto distinct physical CPU cores concurrently.

6 Process Synchronization & The Critical Section Problem

6.1 Independent vs Cooperative Processes

- **Independent Process:** Cannot affect or be affected by the execution of other processes. It shares no state or mutable data.
- **Cooperative Process:** Shares data, memory, or files with other processes. Requires synchronization to avoid inconsistent data states.

6.2 The Critical Section Problem

> **Critical Section (CS):** A segment of code in which a process accesses and manipulates shared resources (such as shared variables, files, or hardware devices). Concurrent execution by multiple processes inside their critical sections without coordination produces **race conditions**.

General Structure of a Synchronized Process

```
do {
    /* ENTRY SECTION: Request permission, acquire locks */

    // CRITICAL SECTION: Access shared variables / hardware
```

```
    /* EXIT SECTION: Release locks, notify waiting processes */

    // REMAINDER SECTION: Non-critical, independent computations
} while (true);
```

6.3 Three Mandatory Criteria for a Valid CS Solution

Any algorithmic or hardware solution to the Critical Section problem must satisfy three strict requirements:

1. Mutual Exclusion

If process P_i is executing in its critical section, then no other processes can be executing in their critical sections simultaneously.

2. Progress

If no process is executing in its critical section and some processes wish to enter, only those processes that are **not** executing in their remainder section can participate in deciding which will enter next, and this selection cannot be postponed indefinitely.

3. Bounded Waiting

There must exist a bound (limit) on the number of times that other processes are allowed to enter their critical sections after a process has made a formal request to enter its critical section and before that request is granted (prevents starvation).

7 Software Solutions for Two Processes

7.1 Algorithm 1: Strict Alternation (Turn Variable)

In Algorithm 1, processes share a single common integer variable `turn`, initialized to `0` or `1`.

Algorithm 1: Structure of Process P_i (where $j = 1 - i$)

```
int turn = 0; // Shared variable

// Process Pi
do {
    while (turn != i); // Busy waiting (Spinlock)

    /* Critical Section */

    turn = j;          // Pass turn to Pj

    /* Remainder Section */
} while (1);
```

Analysis of Algorithm 1

- **Mutual Exclusion:** Satisfied. Since `turn` can be either 0 or 1, only one process can enter its CS at any moment.
- **Progress: VIOLATED.** Execution is tied to strict alternation. If P_0 exits its CS and has no desire to re-enter (remains in remainder section), P_1 enters once, sets turn $= 0$, and when P_1 wishes to re-enter, it is blocked forever waiting for P_0 to change `turn`.

7.2 Algorithm 2: Flag Array (Signaling Intent)

To eliminate strict alternation, Algorithm 1's single turn variable is replaced by a boolean array `flag[2]`, initialized to `false`.

Algorithm 2: Structure of Process P_i (where $j = 1 - i$)

```
boolean flag[2] = {false, false}; // Shared array

// Process Pi
do {
    flag[i] = true;    // Signal intent to enter CS
    while (flag[j]);   // Wait if Pj is also ready or in CS

    /* Critical Section */

    flag[i] = false;   // Reset intent upon exit

    /* Remainder Section */
} while (1);
```

Analysis of Algorithm 2

- **Mutual Exclusion:** Satisfied.
- **Progress: VIOLATED (Deadlock).** If P_0 and P_1 both set `flag[0] = true` and `flag[1] = true` concurrently before reaching the `while` check, both evaluate the condition to `true`, entering an infinite mutual spinlock where neither can progress.

7.3 Algorithm 3: Peterson's Algorithm

Peterson's Solution: Peterson's Algorithm is a classic software-based solution for two processes that combines the key ideas of Algorithm 1 (turn variable) and Algorithm 2 (flag array).

Peterson's Algorithm for Process P_i and Process P_j

```
int turn;
boolean flag[2] = {false, false};

// Process Pi (i = 0, j = 1 or vice-versa)
do {
    flag[i] = true;                // State intent to enter
    turn = j;                      // Be courteous: offer turn to Pj
    while (flag[j] && turn == j);  // Busy wait while Pj wants CS and has
    turn

    /* CRITICAL SECTION */

    flag[i] = false;               // Reset flag on exit

    /* REMAINDER SECTION */
} while (true);
```

Formal Correctness Proof

1. **Mutual Exclusion:** For both P_0 and P_1 to be in their CS simultaneously, both `flag[0]` and `flag[1]` must be `true`, and both `turn == 1` and `turn == 0` must hold simultaneously, which is logically impossible.
2. **Progress:** A process P_i is prevented from entering only if `flag[j] == true` and `turn == j`. If P_j does not want to enter, flag$[j] =$ false, allowing P_i to enter immediately.
3. **Bounded Waiting:** If P_j exits and wishes to re-enter, it must execute turn $= i$, immediately granting P_i entry after at most one turn.

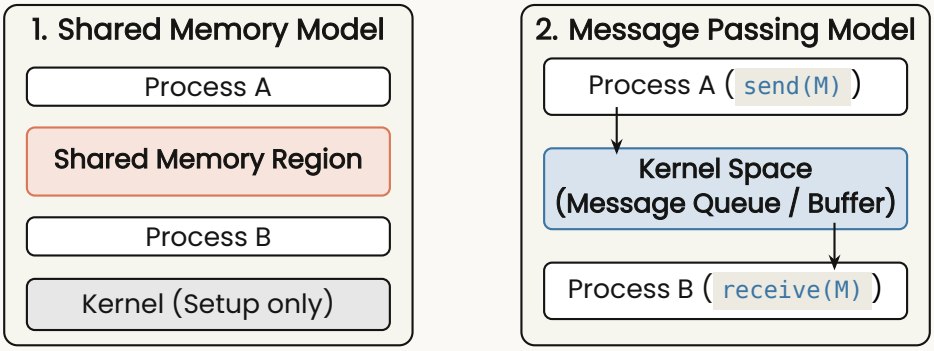
Modern Hardware Limitations of Peterson's Algorithm

Peterson's Algorithm assumes sequential consistency. On modern superscalar multicore architectures, optimizing compilers and out-of-order execution hardware reorder independent memory reads and writes (`flag[i]=true` and `turn=j`). This breaks Peterson's algorithm unless explicit **Memory Barriers / Fences** (`mfence` , atomic variables) are enforced.

8 Inter-Process Communication (IPC)

8.1 Overview and Comparison of IPC Models

Because processes execute in isolated virtual memory address spaces, the operating system provides two foundational IPC mechanisms:



Parameter	Shared Memory Model	Message Passing Model
Mechanism	Address spaces overlap; processes read/write memory directly.	Information exchanged via kernel system calls (`send()` , `receive()`).
Kernel Involvement	Only required during segment setup; zero kernel overhead during data transfer.	Kernel mediates every single message transfer.
Speed & Bandwidth	Maximum memory-bus speed; ideal for massive data sets.	Moderate speed; context switches and buffer copies add latency.
Synchronization	Must be explicitly handled by user application (semaphores, mutexes).	Built into system primitives (blocking/non-blocking).
Distributed Setup	Impractical across network boundaries (without distributed shared memory).	Easily implemented across network links and heterogeneous clusters.

8.2 Message Passing Operations and Communication Link Properties

A message passing facility provides at least two basic primitive operations:

`send(message)` and `receive(message)` (1)

1. Direct vs Indirect Communication

- **Direct Communication:** Senders and receivers explicitly name each other:
  - `send(P, message)` — Send a message directly to Process P.
  - `receive(Q, message)` — Receive a message from Process Q.
  - Links are established automatically between pairs; symmetric addressing.
- **Indirect Communication:** Messages are directed to and retrieved from shared mailboxes or ports (e.g., POSIX message queues with unique integer IDs):
  - `send(A, message)` — Send message to mailbox A.
  - `receive(A, message)` — Receive message from mailbox A.

2. Synchronization Modes (Blocking vs Non-Blocking)

- **Blocking Send (Synchronous):** The sending process is suspended until the message is received by the destination process or mailbox.
- **Non-Blocking Send (Asynchronous):** The sending process deposits the message into the buffer and immediately resumes execution.
- **Blocking Receive (Synchronous):** The receiver blocks until a valid message arrives.
- **Non-Blocking Receive (Asynchronous):** The receiver retrieves either a valid waiting message or an immediate null status.

8.3 Buffering Capacities in Message Passing

Messages exchanged across communication links reside in temporary kernel queues implemented in one of three ways:

Buffering Scheme	Queue Capacity	Operational Behavior
Zero Capacity	Length = 0 messages	Link cannot hold any waiting messages. Sender **must block** until receiver accepts the message (**Rendezvous**).
Bounded Capacity	Finite length = n messages	Holds up to n messages. Sender continues immediately if space exists; sender blocks only if queue is full.
Unbounded Capacity	Infinite length (∞)	Queue is dynamically sized. Sender **never blocks**; messages queue up indefinitely.

9 Summary Review & Exam Booster Cheat Sheet

Key Concepts Quick Summary

- **OS Definition:** Intermediary system software acting as a resource manager and abstract machine.
- **Two OS Components: Shell** (outer user interface / command interpreter) and **Kernel** (core engine managing CPU, memory, devices).
- **PCB Elements:** Pointer, Process State, PID, PC, CPU Registers, Memory Limits, Scheduling Info, Accounting, I/O Status.
- **Process States (5-State Model):** New $\rightarrow$ Ready $\rightleftharpoons$ Running $\rightarrow$ Terminated (with Waiting on I/O).
- **Process Lifecycle Calls:** `fork()` clones process, `exec()` loads program, `wait()` blocks parent, `exit()` terminates child.
- **Schedulers:** Long-Term (Degree of Multiprogramming), Short-Term (CPU assignment, high speed), Medium-Term (Swapper).
- **Threads (LWP):** Share code, data, open files; possess private stack, registers, and program counter.
- **Thread Types:** ULT (fast, user space, whole process blocks on I/O) vs KLT (kernel-managed, multicore capable, non-blocking).
- **Critical Section Criteria:** Mutual Exclusion, Progress, Bounded Waiting.
- **Peterson's Algorithm:** 2-process solution combining `turn` and `flag[2]`; satisfies all 3 criteria (requires memory barriers on modern CPUs).
- **IPC Models:** Shared Memory (fastest, user-synchronized) vs Message Passing (kernel-mediated, distributed-ready).
- **Message Buffering:** Zero (Rendezvous), Bounded (blocks when full), Unbounded (never blocks sender).